# Python 3.12 AsyncIO & High-Performance Coroutines

Presented By isha

# Lecture Agenda & Objectives

- • Core Syntax & Architecture
- • Implementation & Code Walkthrough
- • Hands-on Coding Exercise
- • Best Practices & Pitfalls
- • Summary & Next Steps

# Python 3.12 AsyncIO & High-Performance Coroutines: Concept 1

- Fundamental logic of Python 3.12 AsyncIO & High-Performance
- • Direct state handling and memory
- • Clean modular implementation

```
// PYTHON 3.12.8
# Concept: Python 3.12 AsyncIO & High-Performance Corouti...
message = "Python 3.12 AsyncIO & High-Performance Corouti...
print(message)
return message
asyncio.run(demonstrate_concept())
```

# Introduction to AsyncIO in Python 3.12

- • AsyncIO simplifies asynchronous programming in Python
- • New features for better coroutine management
- • Supports non-

```
// PYTHON 3.12.8
# Concept: Introduction to AsyncIO in Python 3.12
message = "Introduction to AsyncIO in Python 3.12 Execute...
print(message)
return message
asyncio.run(demonstrate_concept())
```

# Introduction to AsyncIO in Python 3.12

- • AsyncIO for non-blocking I/O operations
- • Enhanced concurrency with coroutines
- • Built-in support for asynchronous generators

```
// PYTHON 3.12.8
# Concept: Introduction to AsyncIO in Python 3.12
message = "Introduction to AsyncIO in Python 3.12 Execute...
print(message)
return message
asyncio.run(demonstrate_concept())
```

# Python 3.12 AsyncIO & High-Performance Coroutines: Concept 4

- Fundamental logic of Python 3.12 AsyncIO & High-Performance
- • Direct state handling and memory
- • Clean modular implementation

```
// PYTHON 3.12.8
# Concept: Python 3.12 AsyncIO & High-Performance Corouti...
message = "Python 3.12 AsyncIO & High-Performance Corouti...
print(message)
return message
asyncio.run(demonstrate_concept())
```

# Python 3.12 AsyncIO & High-Performance Coroutines

- • Introduction to AsyncIO in Python 3.12
- • Enhanced support for async/await syntax
- • New features for better coroutine

```
// PYTHON 3.12.8
# Concept: Python 3.12 AsyncIO & High-Performance Coroutines
message = "Python 3.12 AsyncIO & High-Performance Corouti...
print(message)
return message
asyncio.run(demonstrate_concept())
```

# Hands-On Lab: Python 3.12 AsyncIO & High-Performance Coroutines

- Implement the target logic
- • Pass all edge case assertions
- • Observe expected console output
- • Refactor for optimal performance

```
// PYTHON 3.12.8
# Python 312 AsyncIO  HighPerformance Coroutines Example
def compute_result():
    return 'Result: Verified Output'

print(compute_result())
```

# Quick Knowledge Check

- • Fundamental logic of Quick Knowledge Check
- • Direct state handling and memory
- • Clean modular implementation pattern

```
// PYTHON 3.12.8
# Concept: Quick Knowledge Check
message = "Quick Knowledge Check Executed in Python 3.12"
print(message)
return message
asyncio.run(demonstrate_concept())
```

# Python 3.12 AsyncIO & High-Performance Coroutines

- • Introduces new features for async programming
- • Enhances concurrency with improved coroutines
- • Optimizes performance with new asyncio

```
// PYTHON 3.12.8
{
  "async def example_coroutine():\n      await asyncio.sl...
}
```

# Thank You & Questions

- • Interactive Q&A Session
- • Hands-on Code Review
- • Repository Links Shared Online
- • Next Lab Preview